

JAVA 21

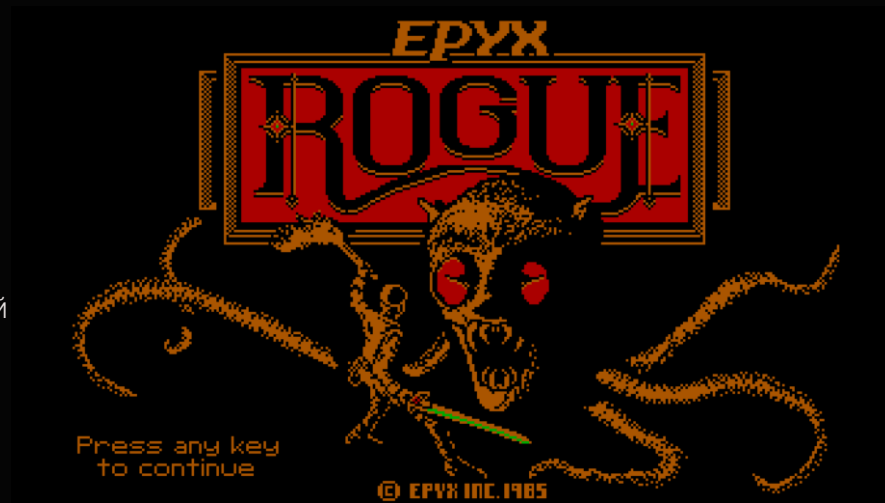
УЧЕБНЫЙ ПРОЕКТ

Roguelike Console Game

Консольная игра в жанре Roguelike на Java 21 — современная реализация классической механики Rogue 1980 года с процедурной генерацией подземелий.

Автор: Булгучев Ибрагим

Группа ИСиТ I-курс



О проекте

Roguelike Console Game — консольная игра, воссоздающая геймплей классического Rogue (1980) с современным техническим подходом. Проект демонстрирует применение архитектурных паттернов и принципов проектирования.

Цель

Реализовать полнофункциональный Roguelike-движок с процедурной генерацией, пошаговой боевой системой и сохранением прогресса.

1

Проектирование архитектуры MVC

Разделение на Presentation, Domain и Data Layer с изоляцией слоёв.

2

Реализация игровой логики

Процедурная генерация, туман войны, система боя и инвентарь.

3

Инфраструктура проекта

Сборка Gradle, контроль версий Git, сериализация через JSON.

Используемые технологии

Каждая технология решает конкретную архитектурную задачу в проекте.



Java 21

Основной язык реализации. Records, pattern matching, sealed classes для моделирования предметной области.



JSON

Формат сериализации состояния игры: характеристики персонажа, инвентарь, прогресс уровней, таблица рекордов.



JCurser

Библиотека консольного ввода-вывода. Рендеринг игрового поля и обработка клавиатурного ввода в терминале.



Gradle

Система сборки и управления зависимостями. Автоматизация компиляции и упаковки JAR-артефакта.



Git и GitLab

Контроль версий и хостинг репозитория. Ведение истории изменений и совместная разработка.

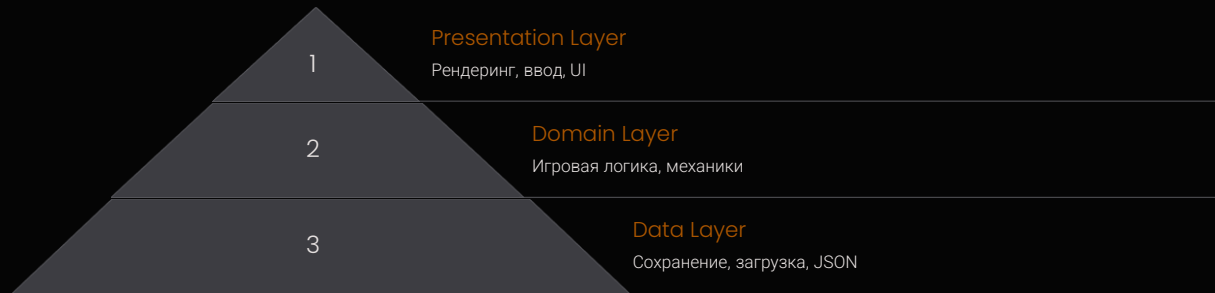


ООП, SOLID, MVC

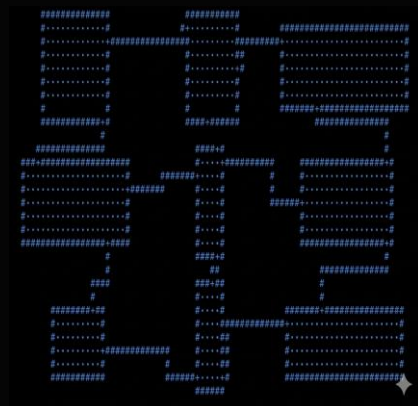
Архитектурные принципы проектирования. Инкапсуляция сущностей, разделение ответственности, изоляция слоёв.

Архитектура приложения

Трёхуровневая архитектура с однонаправленной зависимостью. Каждый слой изолирован и взаимодействует только с соседним через чётко определённые интерфейсы.

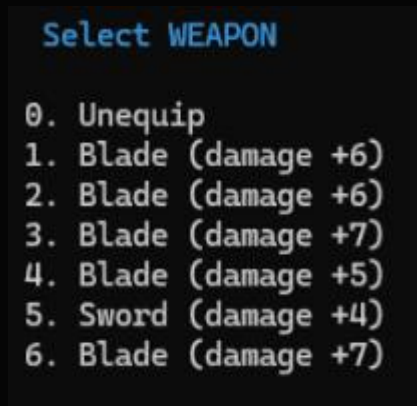


Основные механики



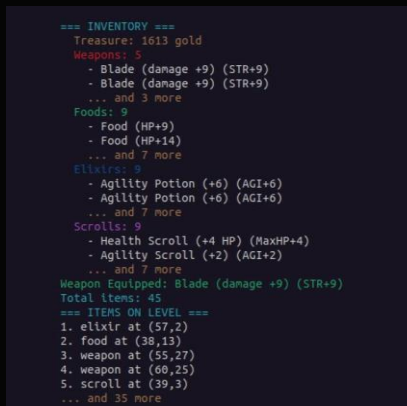
Процедурная генерация

21 уровень подземелий генерируется алгоритмически при каждом запуске. Комнаты, коридоры и размещение врагов уникальны.



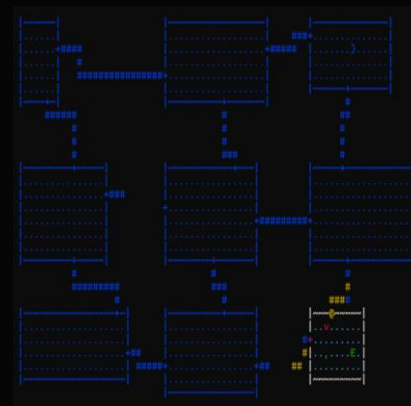
Инвентарь и предметы

Подбор, хранение и использование снаряжения. Оружие, броня, зелья и ключевые предметы.



Система характеристик

Здоровье, сила, ловкость, броня. Ловкость влияет на шанс попадания и уклонения в бою.



Туман войны

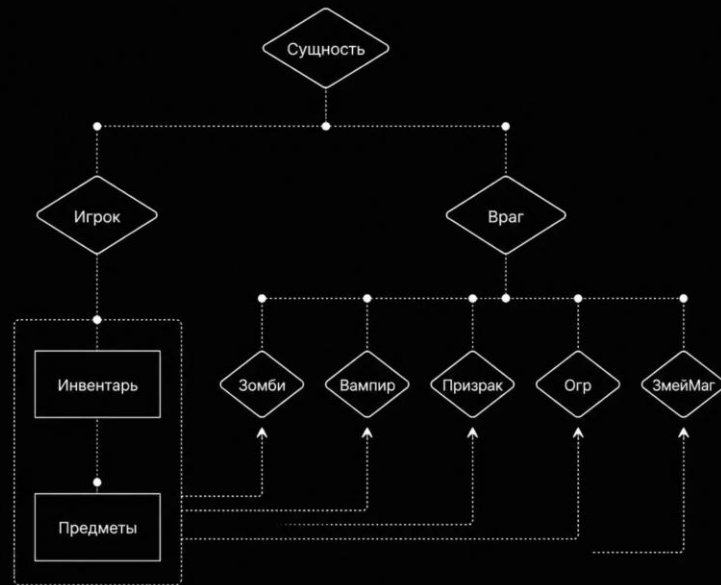
Алгоритм Ray Casting и Брезенхэма определяют видимость клеток. Только исследованные области остаются видны.

Игровые сущности

Каждая сущность — изолированный класс с собственным поведением. Враги наследуют общий интерфейс противников и реализуют специфическое поведение: зомби преследуют игрока, вампиры восстанавливают здоровье при ударе, привидения проходят сквозь стены, ора наносят тяжёлые удары, змеи-маги атакуют на расстоянии.



Все сущности спроектированы по принципу Open/Closed — добавление нового врага не требует изменения существующего кода.



Алгоритмы и технические решения

1 Генерация комнат и коридоров

Алгоритм случайного размещения прямоугольных комнат с последующим соединением коридорами. Проверка пересечений.

2 Проверка связности карты

Обход графа доступных клеток. Гарантия достижимости всех комнат одной из другой.

3 Пошаговая система боя

Игрок и враги действуют поочерёдно. Расчёт попадания по характеристике ловкости атакующего и защитника.

4 Ray Casting и Брезенхэм

Алгоритм Брезенхэма — для расчёта линий обзора. Ray Casting — для тумана войны: определение видимых клеток вокруг игрока.

5 Система сохранений

Сериализация состояния игры в JSON. Сохранение уровня, характеристик, инвентаря и позиции всех сущностей.

```
J App.java x
src > main > java > org > example > J App.java > App
1 package org.example;
2
3 import org.example.domain.GameApplication;
4 import org.example.datalayer.AutosaveRepository;
5 import org.example.datalayer.SessionStatRepository;
6 import org.example.domain.interfaces.IAutosaveRepository;
7 import org.example.domain.interfaces.ISessionStatRepository;
8 import org.example.domain.interfaces.Renderer;
9 import org.example.presentation.views.JCursesRenderer;
10
11 import java.io.IOException;
12
13 public class App {
14
15     public static void main(String[] args) {
16         Renderer renderer = new JCursesRenderer();
17         ISessionStatRepository sessionStatRepository = new SessionStatRepository();
18         IAutosaveRepository autosaveRepository = new AutosaveRepository();
19         try {
20             new GameApplication(renderer, sessionStatRepository, autosaveRepository).run();
21         } catch (IOException e) {
22             throw new RuntimeException(e);
23         }
24     }
25 }
```

Комбинация алгоритмических решений обеспечивает корректную генерацию, видимость и взаимодействие всех объектов игрового мира.

Итоги

Полноценная игра

Реализован консольный Roguelike с процедурной генерацией, сохранениями и боевой системой.

Чистая архитектура

Трёхуровневая MVC с изоляцией слоёв и соблюдением принципов SOLID.

Современный стек

Java 21, Gradle, JSON-сериализация, Git-контроль версий.

Практический опыт

Проектирование сложных программных систем с нуля.
Реализация алгоритмов генерации, обзора и пошагового боя.

GitHub Repository

Исходный код проекта доступен в репозитории:
<https://github.com/nspace0/Rogue-1980-Remake>

ROGUE

